

گزارش پروژه سوم

نرم افزار ریاضی

شماره دانشجویی: ۴۰۲۱۳۴۲۷

تاریخ: تیر ۱۴۰۵

۱ درونیابی لاگرانژ و اسپلین

نقاط داده شده:

(1, 1), (2, 3), (3, 5), (4, 8), (5, 5), (6, 2)

۱.۱ درونیابی لاگرانژ

برای ۶ نقطه، یک چندجمله‌ای درجه ۵ به دست می‌آید. فرمول کلی:

$$P(x) = \sum_{i=0}^5 y_i \cdot L_i(x), \quad L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^5 \frac{x - x_j}{x_i - x_j}$$

کدی که نوشتیم، یک تابع lagrange_basis برای محاسبه هر L_i و یک تابع lagrange_interp برای جمع‌بندی دارد:

```
1 def lagrange_basis(i, x, xn):
2     L = 1.0
3     for j in range(len(xn)):
4         if j != i:
5             L *= (x - xn[j]) / (xn[i] - xn[j])
6     return L
7
8 def lagrange_interp(x, xn, yn):
9     s = 0.0
10    for i in range(len(xn)):
11        s += yn[i] * lagrange_basis(i, x, xn)
12    return s
```

ضرایب چندجمله‌ای (با np.polyfit) به دست آمد:

$$P(x) = 0.175x^5 - 2.9583x^4 + 18.375x^3 - 52.0417x^2 + 68.45x - 31$$

خطای درونیابی در تمام نقاط گره برابر صفر ماشین بود.

۲.۱ اسپلین مکعبی طبیعی

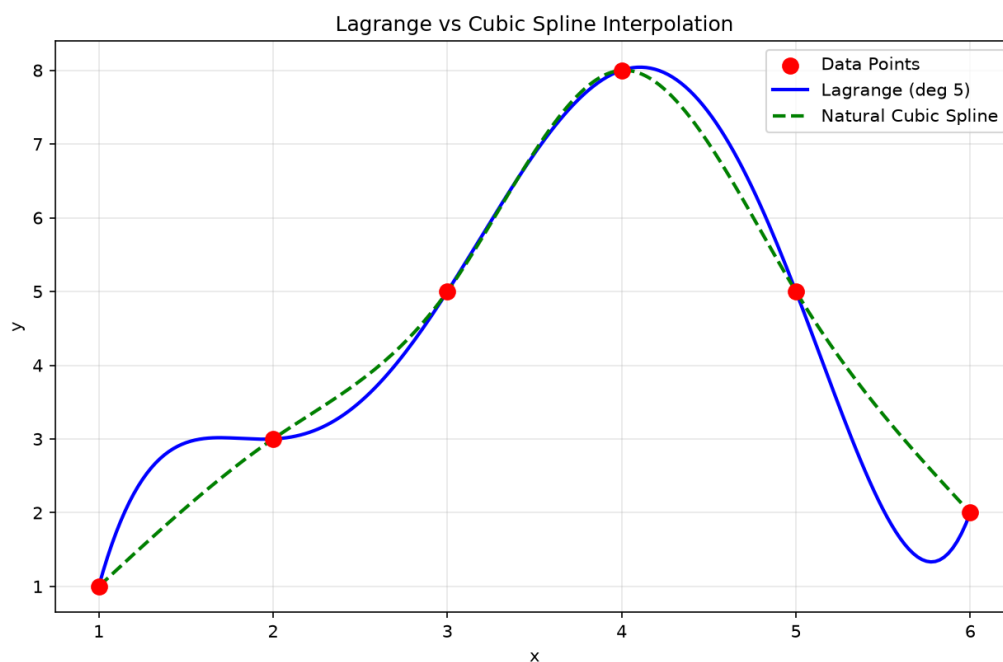
برای اسپلین از کتابخانه scipy استفاده کردم. در این روش به جای یک چندجمله‌ای بزرگ، در هر بازه $[x_i, x_{i+1}]$ یک چندجمله‌ای درجه ۳ جداگانه ساخته می‌شود:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

شرایط مرزی طبیعی یعنی S'' در دو سر بازه صفر باشد. ضرایب هر بازه از خروجی کد:

[1, 2]:	d=-0.1866,	c=0.0000,	b=2.1866,	a=1.0000
[2, 3]:	d= 0.9330,	c=-0.5598,	b=1.6268,	a=3.0000
[3, 4]:	d=-2.5455,	c=2.2392,	b=3.3062,	a=5.0000
[4, 5]:	d= 2.2488,	c=-5.3971,	b=0.1483,	a=8.0000
[5, 6]:	d=-0.4498,	c=1.3493,	b=-3.8995,	a=5.0000

۳.۱ مقایسه دو روش



شکل ۱: مقایسه لاگرانژ و اسپلاین

همان‌طور که در نمودار مشخص است، چندجمله‌ای لاگرانژ دقیقاً از همه نقاط عبور می‌کند ولی بین نقاط نوسان زیادی دارد (مثلاً بین $x = 4$ و $x = 6$ به شدت بالا و پایین می‌رود). اسپلاین مکعبی هم از همه نقاط عبور می‌کند ولی خیلی هموارتر است و نوسان ندارد. برای کاربردهای عملی اسپلاین انتخاب بهتری است.

۲ تحلیل فایل اکسل

فایل اکسل داده شده شامل زمان اجرای ۳ الگوریتم مختلف برای اندازه‌های ۱۰۰ تا ۶۰۰ کیلوبایت است. کد با pandas فایل را می‌خواند و ستون‌ها را به صورت خودکار تشخیص می‌دهد (هیچ داده‌ای دستی وارد نشده):

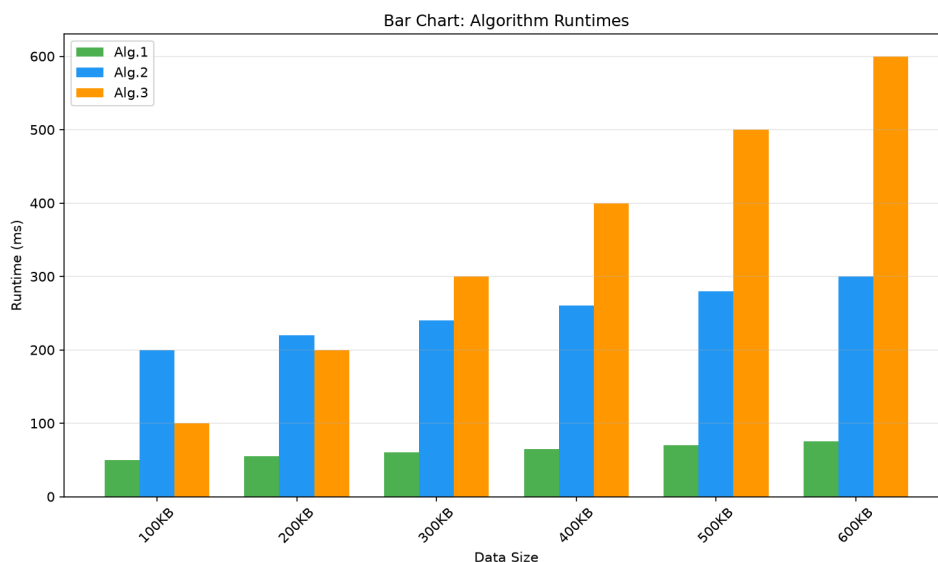
```
1 df = pd.read_excel(excel_file, engine='openpyxl')
2 size_col = df.columns[0]
3 a1, a2, a3 = df.columns[1], df.columns[2], df.columns[3]
```

داده‌های خوانده شده:

اندازه	Alg.۱	Alg.۲	Alg.۳
۱۰۰ KB	۵۰	۲۰۰	۱۰۰
۲۰۰ KB	۵۵	۲۲۰	۲۰۰
۳۰۰ KB	۶۰	۲۴۰	۳۰۰
۴۰۰ KB	۶۵	۲۶۰	۴۰۰
۵۰۰ KB	۷۰	۲۸۰	۵۰۰
۶۰۰ KB	۷۵	۳۰۰	۶۰۰

۱.۲ الف) نمودارها

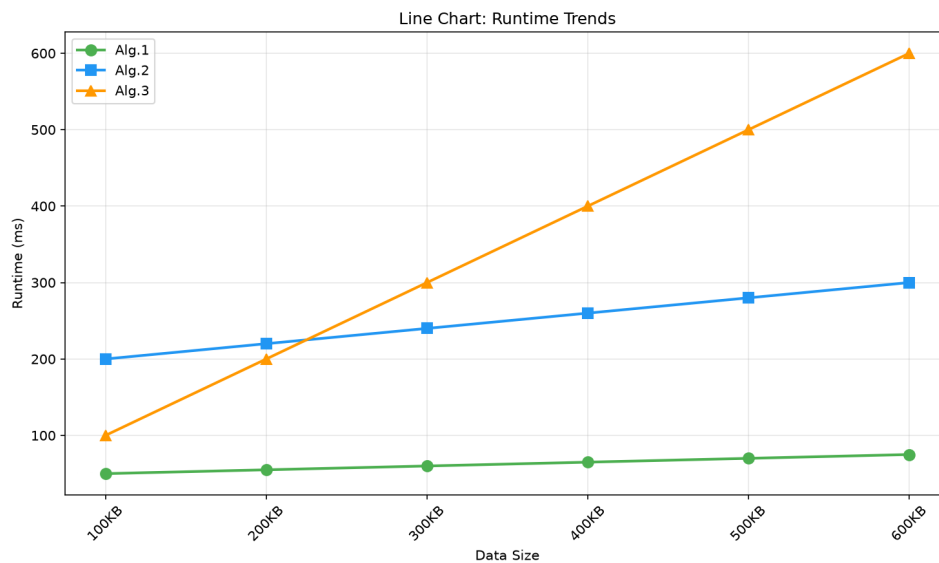
۱.۱.۲ نمودار میله‌ای



شکل ۲: نمودار میله‌ای

Alg.1 سریع‌ترین الگوریتم است و رشد خیلی ملایمی دارد (از ۵۰ تا ۷۵ میلی‌ثانیه). Alg.2 زمان متوسطی دارد ولی رشدش یکنواخت و خطی است. Alg.3 بدترین وضعیت را دارد و از ۱۰۰ به ۶۰۰ میلی‌ثانیه می‌رسد که نشان‌دهنده پیچیدگی بالاتر است.

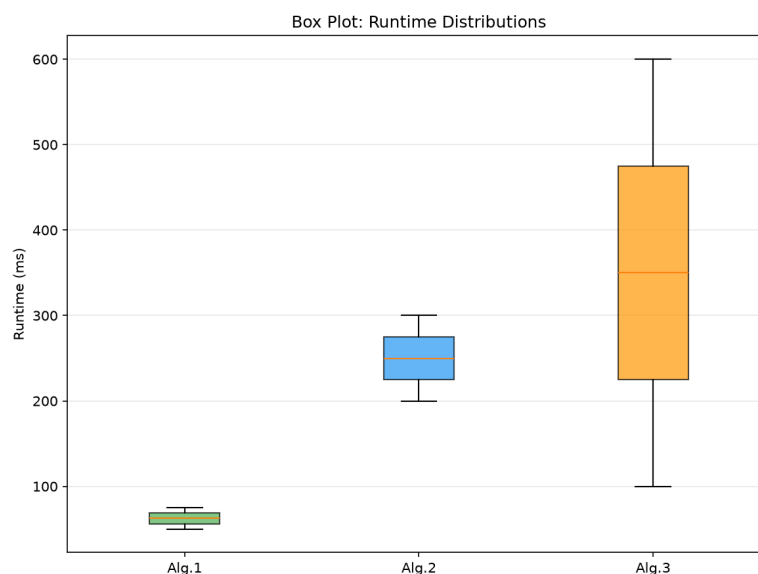
۲.۱.۲ نمودار خطی



شکل ۳: نمودار خطی

هر سه الگوریتم رفتار خطی دارند ولی شیب‌شان خیلی فرق می‌کند. Alg.3 شیب تقریباً ۱ دارد (یعنی با دو برابر شدن داده، زمان هم تقریباً دو برابر می‌شود). Alg.1 شیب خیلی کم دارد و عملاً تفاوت چندانی با افزایش داده نمی‌کند.

۳.۱.۲ نمودار جعبه‌ای

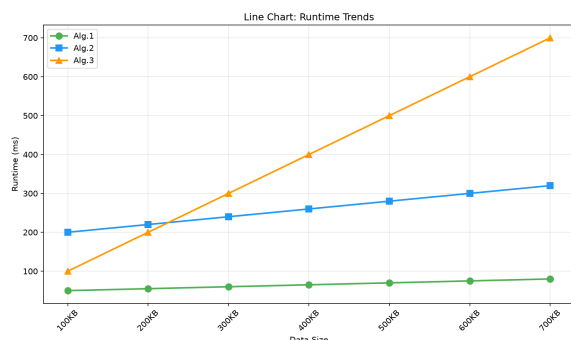


شکل ۴: نمودار جعبه‌ای

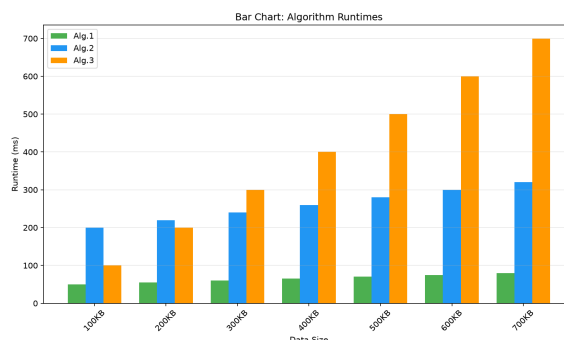
Alg.1 کمترین پراکندگی (IQR کوچک) و Alg.3 بیشترین پراکندگی را دارد. هیچ داده پرتی (outlier) مشاهده نمی‌شود.

۲.۲ (ب) افزودن داده جدید

یک سطر جدید (۷۰۰KB) با مقادیر ۸۰، ۳۲۰، ۷۰۰ به فایل اکسل اضافه شد. بعد کد بدون هیچ تغییری دوباره اجرا شد و نمودارها به صورت خودکار آپدیت شدند:



(ب) خطی آپدیت شده



(آ) میله‌ای آپدیت شده

شکل ۵: نمودارهای بعد از اضافه شدن داده ۷۰۰KB

روند قبلی ادامه دارد و داده ۷۰۰KB هم الگوی خطی را حفظ کرده.

۳.۲ (ج) میانگین الگوریتم ۲

مقادیر Alg.2 برای داده‌های ۱۰۰ تا ۶۰۰ کیلوبایت:

$$[200, 220, 240, 260, 280, 300] \Rightarrow \text{میانگین} = \frac{1500}{6} = 250\text{ms}$$

۳ انتگرال گیری عددی

تابع $f(x) = e^x$ در بازه $[0, 1]$. جواب دقیق:

$$\int_0^1 e^x dx = e - 1 \approx 1.718281828459045$$

۱.۳ الف) سیمپسون و دوزنقه‌ای

هر دو روش را به صورت ترکیبی پیاده‌سازی کردم. مثلاً کد سیمپسون:

```
1 def simpson(f, a, b, n):
2     if n % 2 != 0:
3         n += 1
4     h = (b - a) / n
5     x = np.linspace(a, b, n + 1)
6     fx = f(x)
7     return (h/3) * (fx[0] + fx[-1] + 4*np.sum(fx[1:-1:2]) + 2*np.sum(fx[2:-2:2]))
```

نتایج سیمپسون:

n	تقریب	خطا
۴	۷۱۸۳۱۸۸۴۱۹۲۲.۱	3.70×10^{-5}
۸	۷۱۸۲۸۴۱۵۴۷۰۰.۱	2.33×10^{-6}
۱۶	۷۱۸۲۸۱۹۷۴۰۵۲.۱	1.46×10^{-7}
۳۲	۷۱۸۲۸۱۸۳۷۵۶۲.۱	9.10×10^{-9}
۶۴	۷۱۸۲۸۱۸۲۹۰۲۸.۱	5.69×10^{-10}
۱۲۸	۷۱۸۲۸۱۸۲۸۴۹۵.۱	3.56×10^{-11}

نتایج دوزنقه‌ای:

n	تقریب	خطا
۴	۷۲۷۲۲۱۹۰۴۵۵۸.۱	8.94×10^{-3}
۸	۷۲۰۵۱۸۵۹۲۱۶۴.۱	2.24×10^{-3}
۱۶	۷۱۸۸۴۱۱۲۸۵۸۰.۱	5.59×10^{-4}
۳۲	۷۱۸۴۲۱۶۶۰۳۱۶.۱	1.40×10^{-4}
۶۴	۷۱۸۳۱۶۷۸۶۸۵۰.۱	3.50×10^{-5}
۱۲۸	۷۱۸۲۹۰۵۶۸۰۸۳.۱	8.74×10^{-6}

خطای سیمپسون خیلی سریع‌تر کم می‌شود. مثلاً با $n = 16$ سیمپسون خطای 10^{-7} دارد ولی دوزنقه‌ای خطای 10^{-4} دارد. دلیلش این است که سیمپسون از تقریب درجه ۲ (سه‌می) استفاده می‌کند ولی دوزنقه‌ای از تقریب خطی.

۲.۳ ب) انتگرال گوسی-لژاندر

در این روش به جای نقاط یکنواخت، نقاط بهینه (ریشه‌های چندجمله‌ای لژاندر) انتخاب می‌شوند:

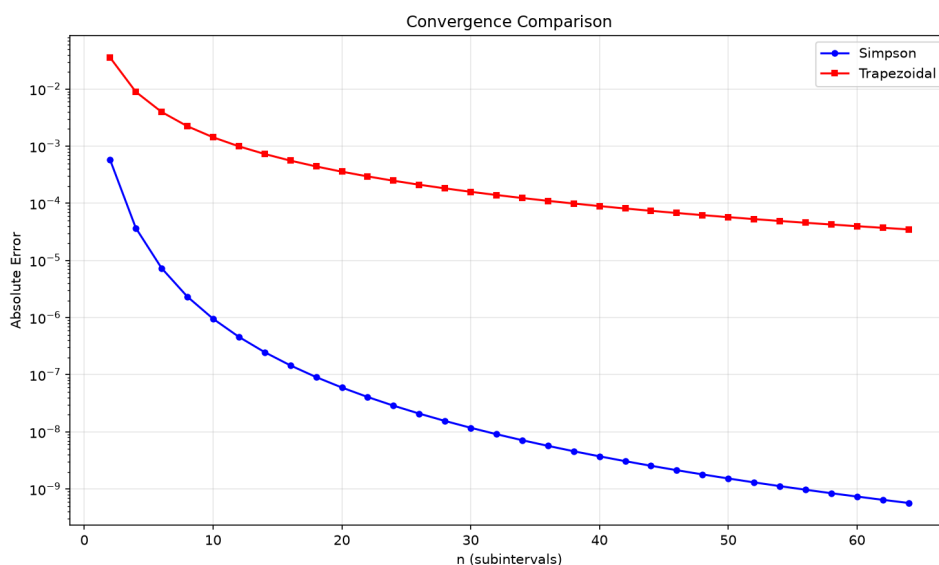
$$\int_a^b f(x) dx \approx \frac{b-a}{2} \sum_{i=1}^n w_i \cdot f\left(\frac{b-a}{2}\xi_i + \frac{a+b}{2}\right)$$

تعداد نقاط	تقریب	خطا
۱	۶۴۸۷۲۱۲۷۰۷۰۰.۱	6.96×10^{-2}
۲	۷۱۷۸۹۶۳۷۸۰۰۸.۱	3.85×10^{-4}
۳	۷۱۸۲۸۱۰۰۴۳۷۳.۱	8.24×10^{-7}
۴	۷۱۸۲۸۱۸۲۷۵۲۶.۱	9.33×10^{-10}
۵	۷۱۸۲۸۱۸۲۸۴۵۸.۱	6.54×10^{-13}
۶	۷۱۸۲۸۱۸۲۸۴۵۹.۱	≈ 0

دقت روش گوسی خیلی بالاتر است. فقط با ۵ نقطه به خطای 10^{-13} می‌رسد — خطایی که سیمپسون حتی با ۱۲۸ زیربازه هم بهش نمی‌رسد.

۳.۳ مقایسه نهایی

روش	تقریب	خطا
سیمپسون ($n = 16$)	1.718281974052	1.46×10^{-7}
دوزنقه‌ای ($n = 16$)	1.718841128580	5.59×10^{-4}
گوسی ($n = 5$)	1.718281828458	6.54×10^{-13}
دقیق	1.718281828459	—



شکل ۶: نمودار همگرایی سیمپسون و دوزنقه‌ای

نمودار همگرایی هم تأیید می‌کند که سیمپسون سریع‌تر همگراست.

۴ لینک گیت‌هاب

تمام کدها و گزارش در مخزن عمومی زیر قرار داده شده:

<https://github.com/FarazFazelifar/Mathematical-Software>